

Support Vector Machines

This notebook is for code reference, please watch the video lecture for a full overview.

Get the Data

We'll use the iris data again since we are already familiar with it.

In [1]:

```
library(ISLR)
```

In [2]:

```
head(iris)
```

Out[2]:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

Building the Model

We'll need the **e1071** library. [More info here.](#)

In [5]:

```
#install.packages('e1071',repos = 'http://cran.us.r-project.org')
```

In [14]:

```
library(e1071)
```

We can use the `svm()` function to build and train a support vector machine model:

In [15]:

```
help(svm)
```

Out[15]:

svm {e1071}	R Documentation
-------------	-----------------

Support Vector Machines

Description

`svm` is used to train a support vector machine. It can be used to carry out general regression and classification (of nu and epsilon-

type), as well as density-estimation. A formula interface is provided.

Usage

```
## S3 method for class 'formula'
svm(formula, data = NULL, ..., subset, na.action =
na.omit, scale = TRUE)
## Default S3 method:
svm(x, y = NULL, scale = TRUE, type = NULL, kernel =
"radial", degree = 3, gamma = if (is.vector(x)) 1 else 1 / ncol(x),
coef0 = 0, cost = 1, nu = 0.5,
class.weights = NULL, cachesize = 40, tolerance = 0.001, epsilon = 0.1,
shrinking = TRUE, cross = 0, probability = FALSE, fitted = TRUE,
..., subset, na.action = na.omit)
```

Arguments

formula	a symbolic description of the model to be fit.
data	an optional data frame containing the variables in the model. By default the variables are taken from the environment which 'svm' is called from.
x	a data matrix, a vector, or a sparse matrix (object of class <code>Matrix</code> provided by the <code>Matrix</code> package, or of class <code>matrix.csr</code> provided by the <code>SparseM</code> package, or of class <code>simple_triplet_matrix</code> provided by the <code>slam</code> package).
y	a response vector with one label for each row/component of <code>x</code> . Can be either a factor (for classification tasks) or a numeric vector (for regression).
scale	A logical vector indicating the variables to be scaled. If <code>scale</code> is of length 1, the value is recycled as many times as needed. Per default, data are scaled internally (both <code>x</code> and <code>y</code> variables) to zero mean and unit variance. The center and scale values are returned and used for later predictions.
type	<code>svm</code> can be used as a classification machine, as a regression machine, or for novelty detection. Depending of whether <code>y</code> is a factor or not, the default setting for <code>type</code> is C-classification or eps-regression, respectively, but may be overwritten by setting an explicit value. Valid options are: • C-classification • nu-classification • one-classification (for novelty detection) • eps-regression • nu-regression
kernel	the kernel used in training and predicting. You might consider changing some of the following parameters, depending on the kernel type. linear: u^*v polynomial: $(\text{gamma} * u^*v + \text{coef0})^{\text{degree}}$ radial basis: $\exp(-\text{gamma} * u - v ^2)$ sigmoid: $\tanh(\text{gamma} * u^*v + \text{coef0})$
degree	parameter needed for kernel of type <code>polynomial</code> (default: 3)
gamma	parameter needed for all kernels except <code>linear</code> (default: $1/(\text{data dimension})$)
coef0	parameter needed for kernels of type <code>polynomial</code> and <code>sigmoid</code> (default: 0)
cost	cost of constraints violation (default: 1)—it is the 'C'-constant of the regularization term in the Lagrange formulation.
nu	parameter needed for <code>nu-classification</code> , <code>nu-regression</code> , and <code>one-classification</code>
class.weights	a named vector of weights for the different classes, used for asymmetric class sizes. Not all factor levels have to be supplied (default weight: 1). All components have to be named.
cachesize	cache memory in MB (default 40)
tolerance	tolerance of termination criterion (default: 0.001)

epsilon	epsilon in the insensitive-loss function (default: 0.1)
shrinking	option whether to use the shrinking-heuristics (default: TRUE)
cross	if a integer value $k > 0$ is specified, a k-fold cross validation on the training data is performed to assess the quality of the model: the accuracy rate for classification and the Mean Squared Error for regression
fitted	logical indicating whether the fitted values should be computed and included in the model or not (default: TRUE)
probability	logical indicating whether the model should allow for probability predictions.
...	additional parameters for the low level fitting function <code>svm.default</code>
subset	An index vector specifying the cases to be used in the training sample. (NOTE: If given, this argument must be named.)
na.action	A function to specify the action to be taken if NAs are found. The default action is <code>na.omit</code> , which leads to rejection of cases with missing values on any required variable. An alternative is <code>na.fail</code> , which causes an error if NA cases are found. (NOTE: If given, this argument must be named.)

Details

For multiclass-classification with k levels, $k > 2$, `libsvm` uses the 'one-against-one'-approach, in which $k(k-1)/2$ binary classifiers are trained; the appropriate class is found by a voting scheme.

`libsvm` internally uses a sparse data representation, which is also high-level supported by the package `SparseM`.

If the predictor variables include factors, the formula interface must be used to get a correct model matrix.

`plot.svm` allows a simple graphical visualization of classification models.

The probability model for classification fits a logistic distribution using maximum likelihood to the decision values of all binary classifiers, and computes the a-posteriori class probabilities for the multi-class problem using quadratic optimization. The probabilistic regression model assumes (zero-mean) laplace-distributed errors for the predictions, and estimates the scale parameter using maximum likelihood.

Value

An object of class "`svm`" containing the fitted model, including:

SV	The resulting support vectors (possibly scaled).
index	The index of the resulting support vectors in the data matrix. Note that this index refers to the preprocessed data (after the possible effect of <code>na.omit</code> and <code>subset</code>)
coefs	The corresponding coefficients times the training labels.
rho	The negative intercept.
sigma	In case of a probabilistic regression model, the scale parameter of the hypothesized (zero-mean) laplace distribution estimated by maximum likelihood.
probA, probB	numeric vectors of length $k(k-1)/2$, k number of classes, containing the parameters of the logistic distributions fitted to the decision values of the binary classifiers ($1 / (1 + \exp(a \cdot x + b))$).

Note

Data are scaled internally, usually yielding better results.

Parameters of SVM-models usually *must* be tuned to yield sensible results!

Author(s)

David Meyer (based on C/C++-code by Chih-Chung Chang and Chih-Jen Lin)
David.Meyer@R-project.org

References

- Chang, Chih-Chung and Lin, Chih-Jen:
LIBSVM: a library for Support Vector Machines
<http://www.csie.ntu.edu.tw/~cjlin/libsvm>
- Exact formulations of models, algorithms, etc. can be found in the document:

Chang, Chih-Chung and Lin, Chih-Jen:

LIBSVM: a library for Support Vector Machines

<http://www.csie.ntu.edu.tw/~cjlin/papers/libsvm.ps.gz>

- More implementation details and speed benchmarks can be found on: Rong-En Fan and Pai-Hsune Chen and Chih-Jen Lin:

Working Set Selection Using the Second Order Information for Training SVM

<http://www.csie.ntu.edu.tw/~cjlin/papers/quadworkset.pdf>

See Also

`predict.svm`, `plot.svm`, `tune.svm`, `matrix.csr` (in package `SparseM`)

Examples

```
data(iris)
attach(iris)

## classification mode
# default with factor response:
model <- svm(Species ~ ., data = iris)

# alternatively the traditional interface:
x <- subset(iris, select = -Species)
y <- Species
model <- svm(x, y)

print(model)
summary(model)

# test with train data
pred <- predict(model, x)
# (same as:)
pred <- fitted(model)

# Check accuracy:
table(pred, y)

# compute decision values and probabilities:
pred <- predict(model, x, decision.values = TRUE)
attr(pred, "decision.values")[1:4,]

# visualize (classes by color, SV by crosses):
plot(cmdscale(dist(iris[, -5])),
     col = as.integer(iris[, 5]),
     pch = c("o", "+")[1:150 %in% model$index + 1])

## try regression mode on two dimensions

# create data
x <- seq(0.1, 5, by = 0.05)
y <- log(x) + rnorm(x, sd = 0.2)

# estimate model and predict input values
m <- svm(x, y)
new <- predict(m, x)

# visualize
plot(x, y)
points(x, log(x), col = 2)
points(x, new, col = 4)

## density-estimation

# create 2-dim. normal with rho=0:
X <- data.frame(a = rnorm(1000), b = rnorm(1000))
attach(X)
```

```
# traditional way:
m <- svm(X, gamma = 0.1)

# formula interface:
m <- svm(~., data = X, gamma = 0.1)
# or:
m <- svm(~ a + b, gamma = 0.1)

# test:
newdata <- data.frame(a = c(0, 4), b = c(0, 4))
predict (m, newdata)

# visualize:
plot(X, col = 1:1000 %in% m$index + 1, xlim = c(-5,5), ylim=c(-5,5))
points(newdata, pch = "+", col = 2, cex = 5)

# weights: (example not particularly sensible)
i2 <- iris
levels(i2$Species)[3] <- "versicolor"
summary(i2$Species)
wts <- 100 / table(i2$Species)
wts
m <- svm(Species ~ ., data = i2, class.weights = wts)
```

[Package *e1071* version 1.6-7]

In [7]:

```
model <- svm(Species ~ ., data=iris)
```

In [9]:

```
summary(model)
```

Out[9]:

```
Call:
svm(formula = Species ~ ., data = iris)
```

```
Parameters:
  SVM-Type:  C-classification
SVM-Kernel:  radial
  cost:      1
  gamma:     0.25
```

```
Number of Support Vectors:  51
```

```
( 8 22 21 )
```

```
Number of Classes:  3
```

```
Levels:
setosa versicolor virginica
```

Example Predictions

We have a small data set, so instead of splitting it into training and testing sets (which you should always try to do!) we'll just score

out model against the same data it was tested against:

In [11]:

```
predicted.values <- predict(model, iris[1:4])
```

In [13]:

```
table(predicted.values, iris[,5])
```

Out[13]:

```
predicted.values setosa versicolor virginica
setosa          50          0          0
versicolor       0          48          2
virginica         0          2         48
```

Just as we've seen before with the iris data set, the setosa values are easily separated with some noise between versicolor and virginica.

Advanced - Tuning

We can try to tune parameters to attempt to improve our model, you can refer to the `help()` documentation to understand what each of these parameters stands for. We use the `tune` function:

In [19]:

```
# Tune for combos of gamma 0.5,1,2
# and costs 1/10 , 10 , 100
tune.results <- tune(svm, train.x=iris[1:4], train.y=iris[,5], kernel='radial',
                    ranges=list(cost=10^(-1:2), gamma=c(.5,1,2)))
```

In [20]:

```
summary(tune.results)
```

Out[20]:

Parameter tuning of 'svm':

- sampling method: 10-fold cross validation

- best parameters:

```
cost gamma
1      0.5
```

- best performance: 0.04666667

- Detailed performance results:

	cost	gamma	error	dispersion
1	0.1	0.5	0.05333333	0.06885304
2	1.0	0.5	0.04666667	0.04499657
3	10.0	0.5	0.05333333	0.05258738
4	100.0	0.5	0.06000000	0.04919099
5	0.1	1.0	0.06666667	0.05443311
6	1.0	1.0	0.06000000	0.06629526
7	10.0	1.0	0.06000000	0.06629526
8	100.0	1.0	0.07333333	0.06629526
9	0.1	2.0	0.10000000	0.06478835
10	1.0	2.0	0.06000000	0.06629526
11	10.0	2.0	0.06000000	0.06629526
12	100.0	2.0	0.06666667	0.07027284

We can now see that the best performance occurs with `cost=1` and `gamma=0.5`. You could try to train the model again with these

specific parameters in hopes of having a better model:

In [21]:

```
tuned.svm <- svm(Species ~ ., data=iris, kernel="radial", cost=1, gamma=0.5)
```

In [22]:

```
summary(tuned.svm)
```

Out[22]:

Call:

```
svm(formula = Species ~ ., data = iris, kernel = "radial", cost = 1,  
    gamma = 0.5)
```

Parameters:

```
SVM-Type: C-classification  
SVM-Kernel: radial  
cost: 1  
gamma: 0.5
```

Number of Support Vectors: 59

```
( 11 23 25 )
```

Number of Classes: 3

Levels:

```
setosa versicolor virginica
```

In [26]:

```
tuned.predicted.values <- predict(tuned.svm,iris[1:4])
```

In [27]:

```
table(tuned.predicted.values,iris[,5])
```

Out[27]:

```
tuned.predicted.values setosa versicolor virginica  
setosa                50          0          0  
versicolor            0          48          2  
virginica              0          2         48
```

Looks like we weren't able to improve on our model! The concept of trying to tune for parameters by just trying many combinations is generally known as a grid search. In this case, we likely have too little data to actually improve our model through careful parameter selection.

Great Job!